CourseCritique: A Comprehensive Course Evaluation System with Analytics and Gamification

Capstone Project Paper

Project By: Gevorg Babajanyan

Degree: Bachelor of Computer Science

Institution: American University of Armenia

Supervisor: Norayr Chilingaryan

Abstract

CourseCritique is a comprehensive course evaluation system redesign, that brings innovation to course evaluation culture and provides with student feedback platform designed to modernize and enhance the course evaluation process in American University of Armenia. The system consists of a React Native mobile application for students and a React web dashboard for administrators/Instructors. The platform enables students to submit structured feedback across three academic phases (Add/Drop, Midterm, Finals) using multi-scale rating system. Meanwhile, administrators can see the aggregated results and analysis of feedbacks, manage courses, and monitor student participation, maintaining feedback anonymity. The system incorporates gamification through achievement badges, biometric authentication for enhanced security, and comprehensive analytics with radar chart visualizations. Built with TypeScript, MobX state management, PostgreSQL database, and JWT-based authentication with refresh token rotation, CourseCritique demonstrates a full-stack solution for collecting and analyzing meaningful student feedback while maintaining user engagement through interactive features.

Keywords: Student Feedback, Course Evaluation, React Native, Gamification, Educational Technology, Data Analytics, Mobile Application

---

Table of Contents

`

- CourseCritique: A Comprehensive Course Evaluation System with Analytics and Gamification
  - Capstone Project Paper
  - Abstract
  - Table of Contents
  - 1. Introduction
  - 2. System Overview
    - 2.1 Mobile Application (Student Interface)
    - 2.2 Web Dashboard (Administrator Interface)
  - 3. Objectives
    - 3.1 Primary Objectives
    - 3.2 Secondary Objectives
  - 4. Scope
    - 4.1 In Scope
    - 4.2 Out of Scope (Future Considerations)
  - 5. System Architecture
    - 5.1 Architecture Diagram
    - 5.2 Component Description
  - 6. Technology Stack and Frameworks
    - 6.1 Frontend (Mobile)
    - 6.2 Frontend (Web Dashboard)
    - 6.3 Backend
    - 6.4 Development & DevOps Tools
    - 6.5 Hosting
    - 6.6 Technology Justification
  - 7. Database Design
    - 7.1 Entity Relationship Diagram (ERD)
    - 7.2 Table Structures
    - 7.3 Database Justification
  - 8. Methodology
    - 8.1 Feedback Aggregation
    - 8.2 Feedback Section Score Calculation
    - 8.3 Bayesian Average (Shrinkage Estimator)
    - 8.5 Course Score Calculation
    - 8.6 Agreement Level Analysis
    - 8.7 Badge Progress Tracking

- 8.8 Open Feedback Aggregation
- 8.9 Refresh Token Validity Period
- 8.10 Notification Trigger Rules

- 9. Backend Implementation
  - 9.1 API Architecture
  - 9.2 Authentication Flow
  - 9.3 Key API Endpoints
  - 9.4 Database Query Optimization
  - 9.5 Error Handling
- 10. Mobile Application Frontend
  - 10.1 Project Structure
  - 10.2 State Management with MobX
  - 10.3 API Client with Interceptors
  - 10.4 Notification System
  - 10.5 Badge System Implementation
  - 10.6 Calendar Integration
  - 10.7 Theme Support
- 11. Web Dashboard
  - 11.1 Project Structure
  - 11.2 Key Features
  - 11.3 Protected Routes
  - 11.4 CORS Configuration
- 12. Security Implementation
  - 12.1 Authentication Security
  - 12.2 Data Protection
  - 12.3 Biometric Authentication
  - 12.4 Refresh Token Protection
- 13. User Interface Design
  - 13.1 Mobile Application UI
  - 13.2 Web Dashboard UI
  - 13.3 Accessibility Considerations
  - 13.4 Responsive Design
- 14. Challenges and Lessons Learned
  - 14.1 Token Refresh Race Conditions
  - 14.2 IOS and Push-Notifications
  - 14.3 Centralized Error Handling
  - 14.4 Key Takeouts
- 15. Future Improvements
  - 15.1 LLM Model For Open-Ended Feedback Processing

- 15.2 Widgets Integration
- 15.3 Phase-Specific Questions.
- 15.4 Push Notifications
- 15.5 Instructor Dashboards
- 15.6 Advanced Analytics
- 15.7 Export Functionality
- 15.8 Bulk import options
- 15.9 Performance Optimization
- 15.10 CI/CD Pipeline

- 16. Conclusion
- 17. References

1. Introduction

Student feedback is a critical component of academic quality assurance in higher education institutions. Traditional paper-based or basic online survey methods often suffer from low student engagement, lack of actionable insights, and minimal student motivation to provide thoughtful responses. CourseCritique addresses these challenges by offering a modern, engaging, and comprehensive feedback collection platform.

The system is designed around the natural academic calendar, with feedback phases aligned to key points in the semester: Add/Drop period (early impressions), Midterm (progress evaluation), and Finals (comprehensive review). This phased approach captures the student experience at different stages, providing instructors and administrators with more nuanced insights than a single end-of-semester survey.

Key innovations of the new system include:

- **Phased feedback collection** – Three distinct evaluation periods (Add/Drop, Midterm, Finals) aligned with the academic semester
- **Multi-scale rating system** – A hybrid approach combining 5-point Likert scales, 3-point scales, and binary (thumbs up/down) questions
- **Directed open feedback** – Structured free-text responses categorized into three actionable areas: advice for future students, course strengths, and improvement suggestions
- **Feedback procession and aggregation** - Development of algorithms to calculate Course Score and evaluate important characteristics of every course on the base of submitted feedbacks and their abundance

Key innovations of the mobile platform include:

- **Gamification** through achievement badges that reward consistent and high-quality feedback submission
- **Biometric authentication** (Face ID / Touch ID) for convenient and secure access
- **Real-time analytics** with radar chart visualizations of aggregated feedback
- **Anonymous feedback display** to protect student privacy while providing valuable course insights

The project demonstrates a full-stack development approach, integrating modern web and mobile technologies with robust backend infrastructure. The following sections detail the system's architecture, implementation decisions, challenges encountered, and lessons learned throughout the development process.

2. System Overview

CourseCritique is divided into two primary interfaces serving different user roles, supported by a unified backend API.

2.1 Mobile Application (Student Interface)

The React Native mobile app provides students with the following capabilities:

1. **Authentication**: Email/password **OR** Username/password login with optional biometric authentication for subsequent sessions
2. **Pending Reminders**: Visual calendar showing open feedback periods with color-coded deadlines
3. **Feedback Submission**: Multi-step forms with four types of rating scales:
   - 5-point Likert scale (Strongly Disagree to Strongly Agree)
   - 3-point scale for specific metrics
   - Binary thumbs up/down for overall satisfaction
   - Guided Open text feedback for free comments(Advice For Future Generations, Strengths,Improvements)
4. **Feedback History**: Chronological display of submitted feedbacks grouped by academic year and semester
5. **Course Search**: Browse courses and view aggregated anonymous feedback statistics/comments
6. **Achievement Badges**: Progress tracking and notification of earned badges(40+) across four categories (milestones, quality, diversity, bonus)
7. **Profile Management**: View/Request to Edit personal information, academic details, app theme and notification preferences

2.2 Web Dashboard (Administrator Interface)

The React web dashboard provides administrative functionality including:

1. **Overview Dashboard**: Key metrics visualization (total feedbacks, completion rates, average ratings)
2. **Course Management**: View all courses with enrollment details, add feedback periods
3. **Course Details Page**: Individual course view with enrolled students list and feedback analytics, including open feedbacks
4. **Analytics Module**: Bar chart/Likert line visualization of course ratings with configurable time limits, as well as valuable insights
5. **Student Management**: View/ Add student profiles with their enrolled courses and feedback history
6. **Anonymous Feedback Review**: Browse submitted feedbacks statistics without student identification

3. Objectives

3.1 Primary Objectives

The primary objective of this project is to design and develop a secure, engaging, and comprehensive course evaluation system that:

1. **Increases Student Participation**: Through gamification, intuitive UI, and capability to browse course feedbacks
2. **Improves Feedback Quality**: By structuring questions meaningfully and encouraging detailed responses with achievement incentives
3. **Provides Actionable Analytics**: Through visual representations including various charts and trend analysis
4. **Ensures Data Privacy**: By ensuring student identity anonymity in displayed feedback and implementing proper authentication
5. **Supports Administrative Oversight**: Through comprehensive dashboards and management tools
6. **Helps Students to Choose Good Courses** - By calculating an individual score for every course and its core aspects based on existing feedbacks and revealing them to students

3.2 Secondary Objectives

Secondary objectives include:

1. Building a cross-platform mobile solution using React Native to target both iOS and Android
2. Implementing secure JWT-based authentication with refresh token rotation
3. Designing a scalable database with proper normalization and indexing using PostgreSQL
4. Creating a modular, maintainable codebase using TypeScript and MobX state management
5. Providing responsive web dashboard with Ant Design Library components

---

4. Scope

4.1 In Scope

The project covers:

- Full authentication system with email/username/password and biometric login

- Three feedback phases (Add/Drop, Midterm, Finals)
- Multi-scale rating system (5-point, 3-point, binary, open text)
- Achievement badge system with 42 badges across four categories
- Mobile calendar view for tracking feedback deadlines
- Algorithm for aggregating feedback results and calculating course score
- Web dashboard with course and student management
- Anonymous feedback viewing for administrators
- Radar chart analytics for course ratings
- Local notification system for deadline reminders

4.2 Out of Scope (Future Considerations)

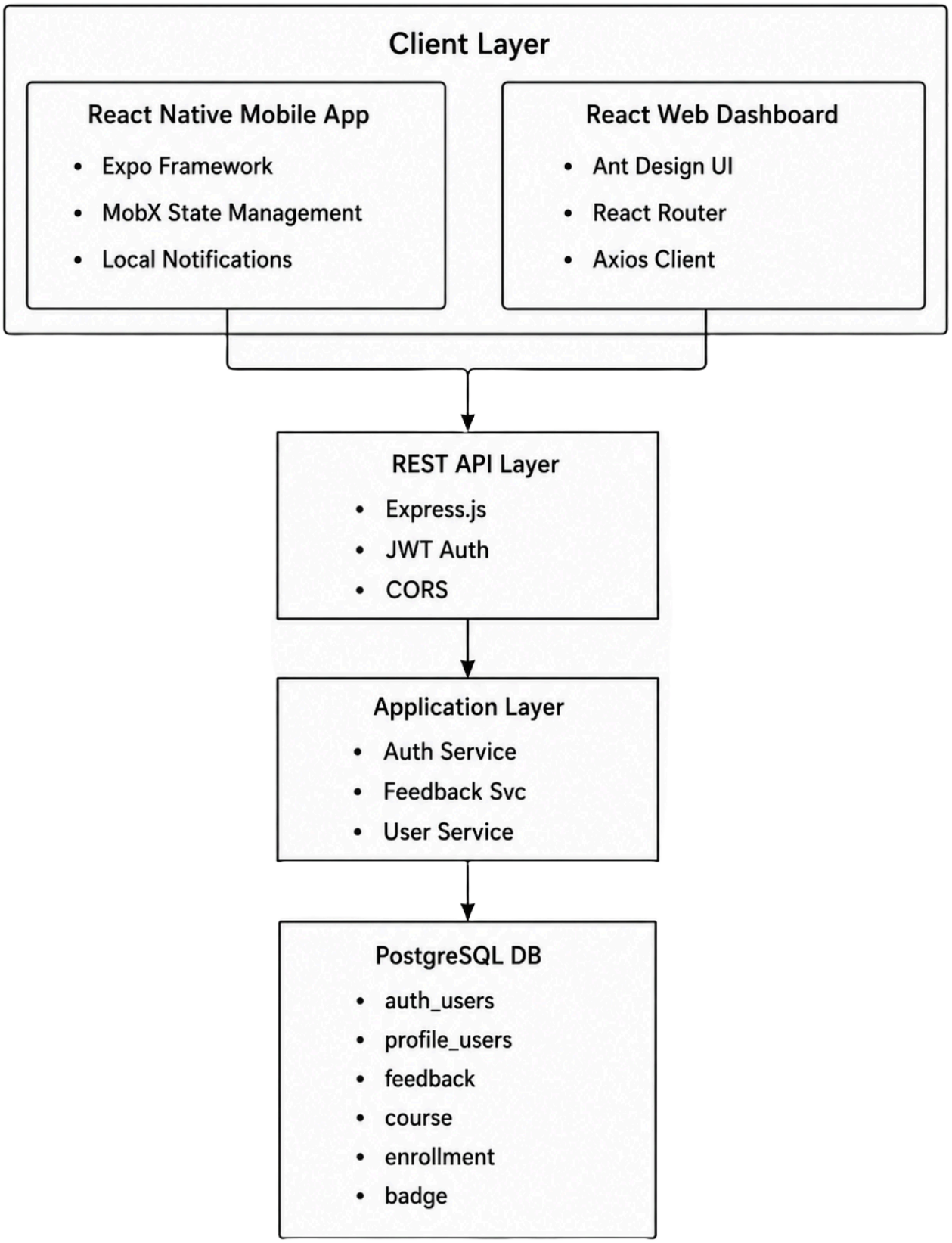
The following features are not included in the current version:

- Push notifications (requires paid Apple Developer Program)
- Instructor-specific dashboards (currently admin-only)
- Student-to-administrator communication system(for handling client-side requests and technical feedback)
- Integration with existing Learning Management Systems (LMS)
- Integration of an LLM model for detailed analytics of open-ended questions
- Historical data import from legacy systems

5. System Architecture

The CourseCritique system follows a modular, three-tier architecture separating frontend presentation, backend application logic, and data persistence.

5.1 Architecture Diagram



5.2 Component Description

Mobile Frontend (React Native / Expo) : Built with Expo framework, utilizing file-based routing with Expo Router. MobX provides reactive state management across authentication, feedback, user profile, notification and settings stores. Local notifications are implemented using Expo Notifications for deadline reminders.

Web Dashboard (React / Ant Design) : Single-page application with protected routes, featuring responsive tables, forms, and interactive charts. Ant Design components have been used to provide a consistent professional appearance.

Backend API (Node.js / Express) : RESTful API with JWT authentication and refresh token rotation. Modular route structure separates concerns for authentication, mobile app and dashboard endpoints. PostgreSQL client for database operations with parameterized queries to prevent SQL injection.

Database (PostgreSQL) : Relational database designed to mock existing model in the system. Implemented tables serve authentication, user profiles, courses, enrollments, badges and feedbacks management purposes.

---

6. Technology Stack and Frameworks

6.1 Frontend (Mobile)

Technology	Version	Purpose
TypeScript	5.0+	Type safety
React Native	0.83.4	Mobile framework
Expo	55.0.18	Development platform
MobX	6.15.0	State management
Axios	1.13.2	HTTP client and interceptors
Expo Router	55.0.7	File-based navigation
Expo SecureStore	55.0.9	Encrypted storage
Expo LocalAuthentication	55.0.9	Biometric login
Expo Notifications	55.0.20	Local notifications
Expo Mail Composer	55.0.13	Sending emails to Dev team about occured errors
React Native Calendars	1.1314.0	Calendar UI

Technology	Version	Purpose
React Native Progress	5.0.1	Progress indicators
React Native Gifted Charts	1.4.76	Display charts about aggregated feedback scores

6.2 Frontend (Web Dashboard)

Technology	Version	Purpose
TypeScript	5.0+	Type safety
React	19.2.0	UI framework
Ant Design	Latest	UI components
React Router	6.x	Routing
Axios	1.13.2	HTTP client
React Icons	Latest	Icon library
Recharts	3.8.1	Displaying Charts

6.3 Backend

Technology	Version	Purpose
Node.js	20.x	Runtime environment
Express	4.18.2	Web framework
PostgreSQL	14+	Relational database
JWT	9.0.0	Authentication tokens
Bcrypt	5.1.0	Password hashing
Cors	2.8.5	Cross-origin requests
Dotenv	16.0.3	Environment configuration

6.4 Development & DevOps Tools

- **Visual Studio Code**: Primary IDE with TypeScript integration
- **Postman**: API testing and documentation
- **Git & GitHub**: Version control
- **Expo Go**: Mobile development testing

- **iOS Simulator / Android Emulator**: Platform testing
- **EAS Build**: Production builds (Android)
- **XCode**: Production builds (IOS)
- **pgAdmin 4**: Database management

6.5 Hosting

- **Database:** Neon
- **Backend API:** Render
- **Dashboard Frontend:** Vercel
- **Mobile app:** Expo Dev

6.6 Technology Justification

React Native was selected over native development to maximize code reuse between iOS and Android platforms while maintaining near-native performance.

MobX was chosen for state management due to its simple, reactive model that integrates seamlessly with React Native components and reduces boilerplate code compared to Redux.

PostgreSQL provides robust relational data support, ACID compliance, and JSONB columns for flexible feedback response storage.

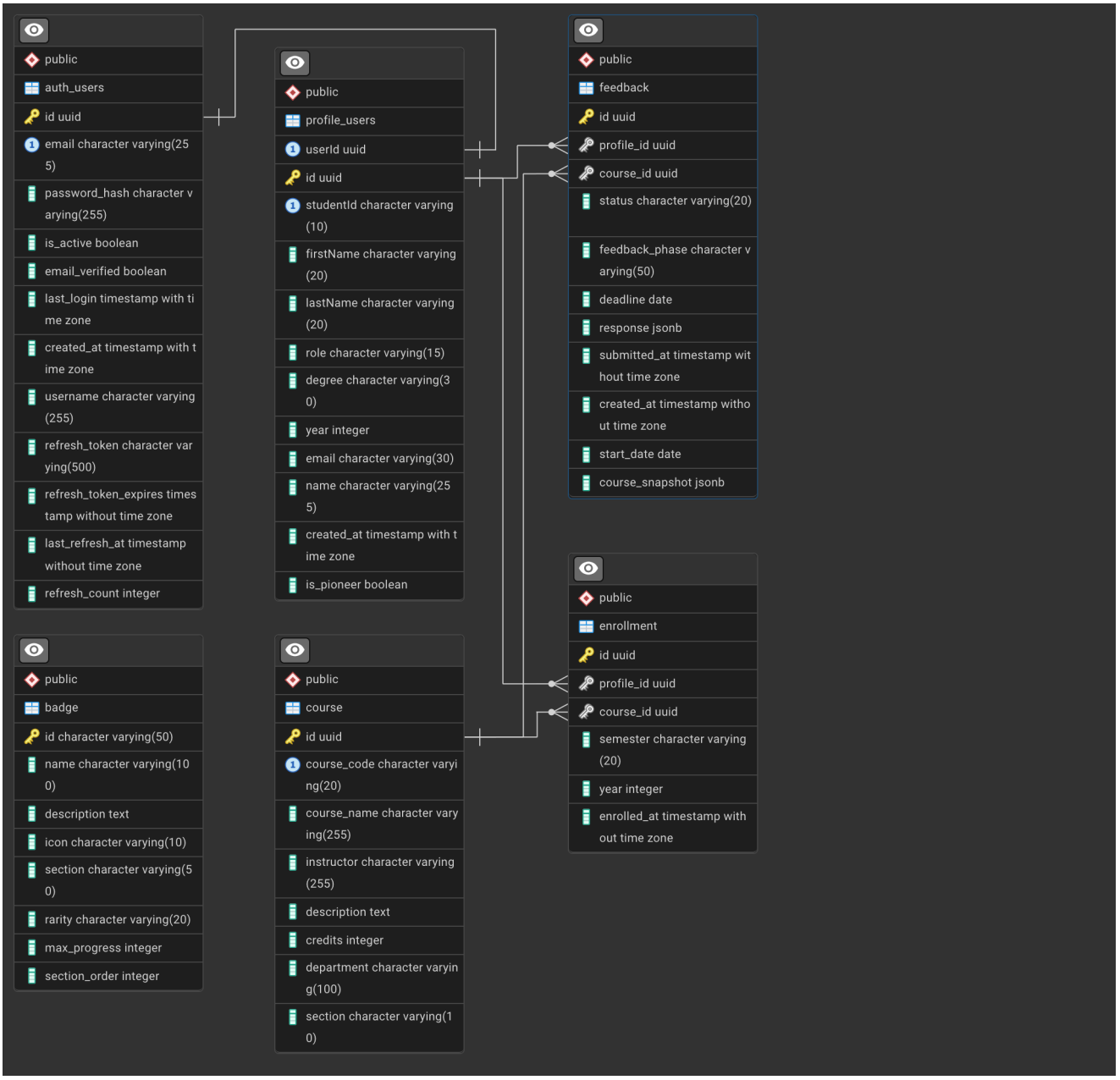
JWT with refresh token rotation balances security and user experience, with short-lived auth tokens and rotation on each refresh to detect token theft.

Expo accelerates development with built-in modules for notifications, secure storage, and biometric authentication without native code configuration.

---

7. Database Design

7.1 Entity Relationship Diagram (ERD)



7.2 Table Structures

auth_users

User authentication credentials, session tokens, and account status management ONLY. This table stores essential login information and security data for all system users.

Column	Type	Description
id	UUID	Primary key
email	VARCHAR(255)	User email (unique)

Column	Type	Description
password_hash	VARCHAR(255)	Bcrypt hash
is_active	BOOLEAN	Account status (default true)
email_verified	BOOLEAN	Email verification status (default false)
last_login	TIMESTAMPTZ	Last login timestamp
created_at	TIMESTAMPTZ	Account creation timestamp
username	VARCHAR(255)	Display/Login username
refresh_token	VARCHAR(500)	Current refresh token
refresh_token_expires	TIMESTAMP	Token expiration timestamp
last_refresh_at	TIMESTAMP	Last token refresh timestamp
refresh_count	INTEGER	Number of token refreshes

profile_users

Stores detailed user profile information including personal and academic details, and role assignments. Links to authentication records and tracks student-specific data like degree programs and pioneer status.

Column	Type	Description
id	UUID	Primary key
firstName	VARCHAR(20)	User's first name
lastName	VARCHAR(20)	User's last name
role	VARCHAR(15)	User role (student/instructor/admin)
degree	VARCHAR(30)	Academic degree program
year	INTEGER	Academic year
email	VARCHAR(30)	Profile email address
studentId	VARCHAR(10)	Student ID number (unique)
userId	UUID	Foreign key to auth_users
name	VARCHAR(255)	Generated full name (firstName + lastName)
created_at	TIMESTAMPTZ	Profile creation timestamp
is_pioneer	BOOLEAN	Pioneer status flag (default false)

course

Maintains the academic course catalog including course codes, credits, instructor names, and departmental information. Each course entry represents a distinct academic offering.

Column	Type	Description
id	UUID	Primary key
course_code	VARCHAR(20)	Course code (unique)
course_name	VARCHAR(255)	Full course name
instructor	VARCHAR(255)	Course instructor name
description	TEXT	Course description
credits	INTEGER	Credit hours (1-6)
department	VARCHAR(100)	Academic department
section	VARCHAR(10)	Course section (default 'A')

enrollment

Junction table between user_profile and course tables to implement many-to-many relationship. Tracks which students are enrolled in which courses for specific semesters and academic years.

Column	Type	Description
id	UUID	Primary key
course_id	UUID	Foreign key to course
section	VARCHAR(10)	Enrollment section
semester	VARCHAR(20)	Semester (Fall/Spring/Summer)
year	INTEGER	Academic year
enrolled_at	TIMESTAMP	Enrollment timestamp (default now)
profile_id	UUID	Foreign key to profile_users

feedback

Collects student feedback for courses across different evaluation periods (add/drop, midterm, finals). Stores both the submitted responses and a snapshot of course data at the time of feedback.

Column	Type	Description
id	UUID	Primary key
course_id	UUID	Foreign key to course
status	VARCHAR(20)	pending/completed (default 'pending')
feedback_phase	VARCHAR(50)	addDrop/midterm/finals
deadline	DATE	Submission deadline
response	JSONB	Feedback responses in JSON format
submitted_at	TIMESTAMP	Submission timestamp
created_at	TIMESTAMP	Record creation timestamp (default now)
start_date	DATE	Feedback period start date
profile_id	UUID	Foreign key to profile_users
course_snapshot	JSONB	Snapshot of course data at feedback time

Each of the responses are JSON objects containing following information:

```
FeedbackRatings = {
#Section 1: Course Design
course_pace: Rating3;
course_load: Rating3;
class_organization: Rating3;

#Section 2: Materials
course_materials: Rating3;
assignment_instructions: Rating5;
grading_rubrics: Rating3;

#Section 3: Engagement
class_management: Rating5;
student_participation: Rating3;
in_class_queries: Rating5;
concern_learning: Rating5;

#Section 4: Support
availability: Rating3;
feedback_on_assignments: Rating3;
inspires_motivation: Rating3;
```

```
#Section 5: Learning Outcomes
substantial_learning: Thumb;
take_another_course: Thumb;

advice_future_gen?: string;
strengths?: string;
improvements?: string;
};
```

There are three implemented rating scales, that eventually get numeric values:

```
Rating5 = 1 | 2 | 3 | 4 | 5 ;

Rating3 = 1 | 2 | 3 ;

Thumb = 0 | 1 ;
```

Each of those respectively, for user experience and clarity purposes, is represented as a certain value to a user:

```
FEEDBACK_VALUES_BY_TYPE = {

"5": {
1: "Strongly Disagree",
2: "Disagree",
3: "Neutral",
4: "Agree",
5: "Strongly Agree",
},

"3": {
1: "Needs Improvement",
2: "Satisfactory",
3: "Excellent"
},

thumb: {
0: "Disagree",
1: "Agree"
},

}
```

badge

Table badge defines the badge system for gamification, including rarity levels, categories, and progress requirements. Badges are awarded to users for completing milestones, maintaining quality, achieving diversity, or earning bonuses.

Column	Type	Description
id	VARCHAR(50)	Primary key
name	VARCHAR(100)	Badge display name
description	TEXT	Badge description
icon	VARCHAR(10)	Icon identifier
section	VARCHAR(50)	Category: milestones/quality/diversity/bonus
rarity	VARCHAR(20)	Rarity: common/rare/epic/legendary
max_progress	INTEGER	Maximum progress required for badge
section_order	INTEGER	Display order within section

7.3 Database Justification

PostgreSQL was selected for its excellent support for relational data, ACID compliance, and particularly for JSONB columns. The JSONB type is valuable for the `feedback.response` column, allowing flexible storage of rating data across different question sets without requiring schema migrations. JSONB also supports efficient indexing and querying of nested data for analytics.

```
SELECT
  department,
  feedback_phase,
  COUNT(*) as response_count,

  -- Course Structure Metrics (Rating3: 1-3 scale)
  ROUND(AVG((response->>'course_pace')::int), 2) AS avg_course_pace,
  ROUND(AVG((response->>'course_load')::int), 2) AS avg_course_load,
  ROUND(AVG((response->>'class_organization')::int), 2) AS
avg_class_organization,

FROM feedback f
  JOIN course c ON f.course_id = c.id
  WHERE f.status = 'completed'
  AND f.response IS NOT NULL
```

```
GROUP BY department, feedback_phase
ORDER BY department, feedback_phase;
```

Foreign key constraints with `ON DELETE SET NULL` and `ON DELETE CASCADE` ensure data integrity while preserving historical feedback even when courses or users are removed from active systems.

With PostgreSQL's mechanism of triggers and trigger functions it is possible for users, both students and faculty, save and review course feedbacks on courses that get deleted from active courses catalogue - trigger saves a snapshot of course info for any feedback that is addressed to it in the moment of deletion.

---

8. Methodology

This section aims to reveal statistical methods and calculations used throughout the CourseCritique platform, particularly in feedback aggregation, Course Score and corresponding section scores calculation, to ensure meaningful interpretation of feedback data.

8.1 Feedback Aggregation

When designing the Mobile app, one had to come up with means of processing all the feedback data with all its various scales in a way so processed information was both readable and informative to user when displaying it. And if in case of open-ended questions naive approach, randomly revealing a response for each question, worked just as fine, numerically scaled questions required more attention. Naive approach was to take all the courses and compute average response for each question and show that to user:

```
SELECT
  department,
  feedback_phase,
  COUNT(*) as response_count,

  -- Course Structure Metrics (Rating3: 1-3 scale)
  ROUND(AVG((response->>'course_pace')::int), 2) AS avg_course_pace,
  ROUND(AVG((response->>'course_load')::int), 2) AS avg_course_load,
  ROUND(AVG((response->>'class_organization')::int), 2) AS
avg_class_organization,
  ....
```

But with this scenario one quickly finds themselves out confused and overwhelmed every time they open course feedback modal/page, because of the abundance of the

questions(15).

8.2 Feedback Section Score Calculation

Looking at questions, as well as when designing feedback form page, feedback questions appeared to be subject to group in several categories, even more informative than "course" and "instructor". For better user experience, they were distributed between five key topics(Sections): Course Design, Materials, Engagement, Support and Learning Outcomes(detailed distribution in 7.2, feedback). Partially thanks to that design, a decision was made to deliver to user 5 scores instead of 15 - visually cleaner, way easier to interpret.

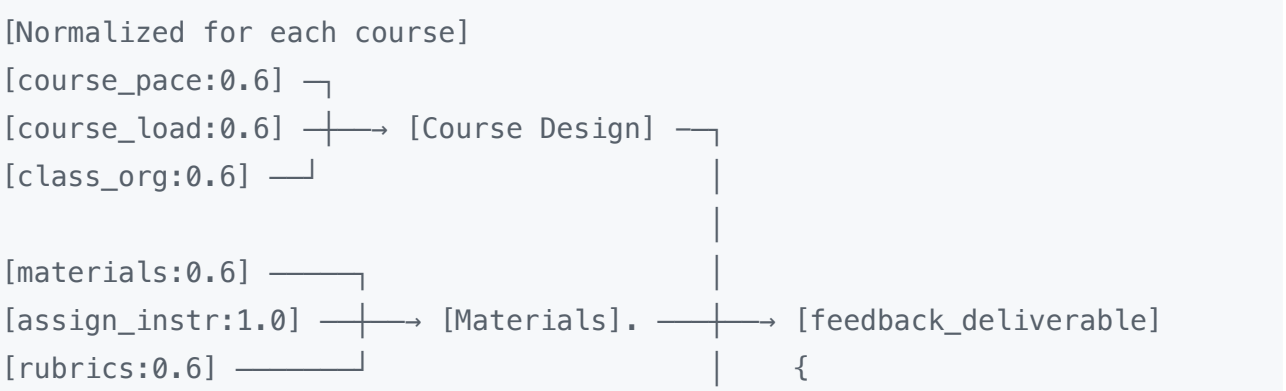
```
FEEDBACK_DELIVERABLE = {
course_design: ["course_pace", "course_load", "class_organization"],
materials: ["course_materials", "assignment_instructions",
"grading_rubrics"],
engagement: [
    "class_management",
    "student_participation",
    "in_class_queries",
    "concern_learning",
],
support: ["availability", "feedback_on_assignments",
"inspires_motivation"],
outcomes: ["substantial_learning", "take_another_course"],
}
```

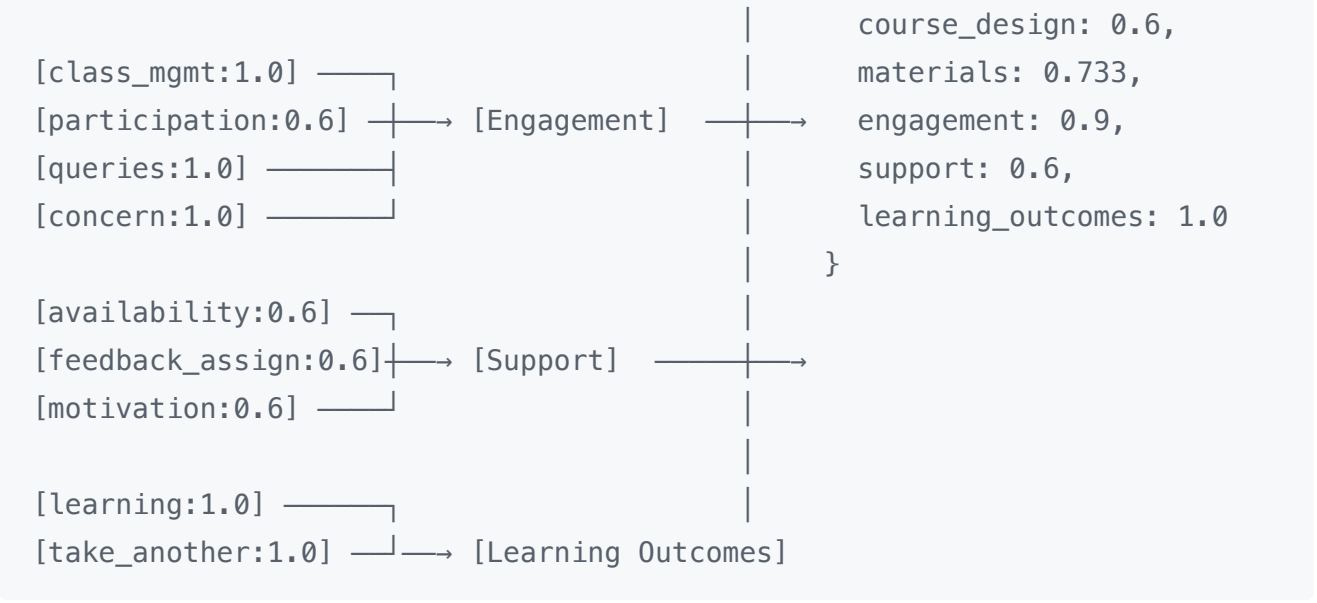
For that purpose, there was a necessity to bring all the course responses to one scale, normalize them.

$normalized[key] = (value - minValue)/(maxValue - minValue)$;

After normalization, for simplicity, an arithmetic mean was taken from every group of questions

`Section Score = (Σnormalised_value_per_section)/n`





Then, after having a dataset of 5 parametered variables for each course, one could use all their analytical skills to analyze student feedbacks of a given course throughout timeline, but for user simplicity, only one measurement was chosen.

8.3 Bayesian Average (Shrinkage Estimator)

When analyzing different feedbacks, one can come to conclusion that taking solely the average value for each section was not enough: two courses with various sample sizes but coinciding means would be treated equally, although, in everyday practice, that's not the case - we trust more to a survey based on larger samples.

```
Course A: [5,5,5] → 1.0
Course B: [5 × 300] → 1.0
```

To address the issue of small sample sizes (courses with few feedbacks), the platform employs a **Bayesian average** (also known as a shrinkage estimator) for radar chart dimensions.

Formula:

```
Bayesian Score = (Prior_Mean × Prior_Weight) + (Sample_sum) /
(Weight_Global + n)
```

Where:

- `Prior_Mean` is the global average score for that dimension across **all courses**
- `Prior_Weight` is a constant (default = 5) representing the confidence in the global average
- `Sample_Sum` is the sum of the scores for that dimension within the specific course
- `n` is the number of completed feedbacks for the course(Sample size)

Interpretation:

- For courses with few feedbacks (n < 5), the score is "shrunk" toward the Prior mean(Global Average)
- For courses with many feedbacks, the course-specific mean dominates(Local Average)
- This prevents courses with only 1–2 feedbacks from appearing at extreme ends of the radar chart

Example: Global average for "Course Pace" is 4.2. Course A has 2 feedbacks averaging 5.0. Bayesian score = `(4.2×5 + 5.0×2)/(5+2) = 4.43` (instead of 5.0)

To compute Bayesian averages for sections, values 0.5(default value is in the center) and 8(prior assumption for smallest sample size) were chosen for prior mean and prior weight correspondingly.

On user end, Section scores are scaled to 10 to increase readability.

$$SectionScore = 1 + bayesian * 9$$

This way, the Bayesian shrinkage estimator provides fair course comparisons even when enrollment numbers vary significantly.

It goes without mentioning , that in this way, values lay mainly on interval 5.5 - 9.5 which is close to real-world rating practices:

- **IMDb:** almost no movies > 9.5
- **Steam:** most games cluster tightly
- **Restaurant ratings:** Majority of reviews is 3.5–4.8

8.4.1 Dashboard Measurments

For dashboard, in order to explore feedback results on deeper(questions') level and make more detailed conclusions, more measurements are used alongside Bayesian means of sections: Arithmetic mean, standard deviation, variance.

8.5 Course Score Calculation

After having all the feedback for a course aggregated in 5 values, nothing holds back calculating one final score for a course - CCScore(CourseCritiqe Score)

```
CCScore =
0.25 * course_design.bayesian +
0.25 * materials.bayesian +
0.25 * engagement.bayesian +
0.15 * support.bayesian +
0.1 * outcomes.bayesian;
```

***Disclaimer:** weights for sections have been chosen based on personal bias of the author and their experience within the course evaluation system. *

8.6 Agreement Level Analysis

Dashboard, apart from providing CCScore and sections' scores, also provides graphs(likert lines and bar plots) and valuable insights for each individual question.
One of those insights is Agreement Level analysis, which, based on response distribution and variance, in some sense interprets charts and tells user(instructor) in what scale the responses to certain question coincide.

```
if (normalizedVariance < 0.33) {
return `High Agreement – ${consensus} `;
} else if (normalizedVariance < 0.67) {
return "Mixed Opinions";
} else {
return "Polarized";
}
```

8.7 Badge Progress Tracking

Badges are earned based on cumulative student activity. Progress toward each badge is calculated as:

Formula for Progress-Based Badges:

```
Progress (%) = min(100, (Current_Count / Target_Threshold) × 100)
```

Example: The "Feedback Veteran" badge requires 25 completed feedbacks. A student with 12 feedbacks has `(12/25) × 100 = 48%` progress.

Rules for Different Badge Types:

Badge Category	Calculation Logic	Example
Milestone (count-based)	Earned when total feedbacks ≥ threshold	"Feedback Master" at 50 feedbacks
Quality (text length)	Earned when number of detailed comments ≥ threshold	"Detail Oriented" requires 5+ comments >574 characters
Streak (consecutive semesters)	Counts consecutive semesters with ≥1 feedback	"Consistent Contributor" at 3 consecutive semesters

Badge Category	Calculation Logic	Example
Diversity (department count)	Earned when distinct departments in feedbacks ≥ threshold	"Explorer" at 5 different departments
Bonus	Specific condition met (e.g., submission after midnight)	"Midnight Owl" requires at least one submission between 12am–5am

8.8 Open Feedback Aggregation

8.8.1 Mobile App

Open-ended responses ("advice_future_gen", "strengths", "improvements") are displayed as randomized quotes in the course statistics modal. For each category, the system:

1. Collects all non-null responses from completed feedbacks
2. On modal open, selects a **random quote** from the collection
3. Provides a refresh button to cycle through other quotes

This approach protects student anonymity while still conveying qualitative insights.

8.8.2 Dashboard

Unlike Mobile app users, Admins and/or Instructors are able to view all feedbacks of given type at once, without need to loop through. In this scenario, as well, student anonymity is preserved.

8.8.3 Important notice

One of the most prioritized future plans of the project refers to redesign of how open-ended feedback is delivered to users, both to students and faculty, particularly by integrating a LLM model to process all feedbacks and provide insights like commonly used descriptions, comparative analysis, descriptions(positive/negative) that started/stopped appearing in recent feedbacks, etc. . The reasoning of leaving that action to the future is the project's focus on analysis of numerical part of feedbacks, necessity of which is based on feedbacks by some instructors anonymity of whose we intend to preserve:)

8.9 Refresh Token Validity Period

The authentication system uses two token types with different lifespans:

Token Type	Validity Duration	Storage Location	Refresh Behavior
Access Token (JWT)	15 minutes	In-memory only	Automatically refreshed on 401 error
Refresh Token (JWT)	30 days	SecureStore (encrypted)	Rotated on each refresh request

Security Justification: Short-lived access tokens limit exposure, while refresh tokens enable seamless user experience. Rotation prevents replay attacks and enables theft detection.

8.10 Notification Trigger Rules

Local notifications are scheduled based on the following rules:

Notification Type	Trigger Timing	Condition
Deadline Reminder	Day of deadline at 9:00 AM	Feedback still pending
Last Chance	24 hours before deadline at 5:00 PM	Feedback still pending
Early Bird	On start date at 10:00 AM	Feedback period opens
Weekly Reminder	Every Monday at 10:00 AM	Any pending feedbacks exist
Achievement Earned	Immediately when badge progress completes	User has earned a new badge

Notifications are **only scheduled** if the user has enabled in-app notifications in settings and has granted notification permissions.

---

9. Backend Implementation

9.1 API Architecture

The backend follows RESTful principles with modular route organization:

```
backend/
├── server.js                # Main entry point
```

```
├── db-config.js              # Database connection
├── package.json
├── .env
│
├── routes/                   # API endpoints
│   ├── mobileApp/            # Mobile App routes
│   │   ├── auth.js           # Authentication
│   │   ├── courses.js        # Course-related
│   │   ├── feedback.js       # Feedback retrieval/submission
│   │   └── profile.js        # User profile
│   └── dashboard/            # Admin dashboard routes
│       ├── course.js
│       ├── dashboard.js
│       └── students.js
│
├── middleware/               # Express middleware
│   ├── verifyToken.js        # JWT authentication
│   └── roleCheck.js          # Role-based access control
│
├── constants/                # Static values
│
├── util/                     # Helper functions
│   ├── feedbackStats.js      # Statistics calculations
│   └── util.js               # General utilities
│
└── node_modules/
```

9.2 Authentication Flow

The authentication system implements JWT with refresh token rotation:

1. **Registration:** The mobile application does not offer self-registration. This design aligns with the platform's closed-user model, which restricts access to currently enrolled university students. Consequently, account creation is performed exclusively by administrators through the web dashboard. Students receive pre-allocated credentials and are empowered to modify them after their initial login.
2. **Login**: User submits email/password → server validates → generates `authToken` (15 min) and `refreshToken` (30 days) → stores refresh token in database → returns both to client
3. **Refresh**: Client sends expired auth token → receives 401 → sends refresh token → server validates and generates new token pair → updates database → returns new tokens

4. **Logout**: Client sends refresh token → server invalidates token in database

Refresh Token Rotation Security:

- Each refresh generates a new refresh token
- Previous refresh tokens become invalid immediately
- If a token is reused, server detects potential theft and invalidates all user tokens

9.3 Key API Endpoints

Method	Endpoint	Description	Auth
POST	/auth/user_login	User authentication	No
POST	/auth/refresh	Token refresh	No
GET	/courses	List all courses	Yes
GET	/courses/:id/stats	Course statistics	Yes
GET	/feedback/pending	User's pending feedbacks	Yes
GET	/feedback/completed	User's completed feedbacks	Yes
POST	/feedback/submit	Submit feedback	Yes
GET	/dashboard/stats	Admin overview	Yes
GET	/dashboard/students	List all students	Yes
POST	/courses/:id/feedback-periods	Create feedback period	Yes

9.4 Database Query Optimization

- Indexed foreign key columns (`user_id`, `course_id`, `profile_id`) for join performance
- Partial index on `status` for pending feedback queries
- JSONB operations for rating aggregations use `->>` and `::numeric` casting

9.5 Error Handling

Centralized error handling middleware both on backend and on frontends catches exceptions and returns consistent error responses:

```
try {

  // operation
```

```
} catch (error) {
  console.error("methodName error:", error.response?.data || error);
  res.status(500).json({ error: "Failed to fetch data" });
}
```

During authorization processes everything is particularly monitored and in case of error all changes made into the database rollback:

```
try {
    await pool.query("BEGIN");

  // operation

    await pool.query("COMMIT");
} catch (error) {
    await pool.query("ROLLBACK");
    console.error("methodName error:", error.response?.data || error);
    res.status(500).json({ error: "Failed to fetch data" });
}
```

On Mobile app, there in case of caught error, user is informed and has option to report it to developer team via email.

Placeholder

---

10. Mobile Application Frontend

10.1 Project Structure

```
mobile-app/
├── app/                              # Expo Router file-based routes
│   ├── (protected)/                  # Protected routes requiring auth
│   │   ├── (tabs)/                   # Tab navigation group
│   │   │   ├── (home)/               # Calendar dashboard
│   │   │   │   └── feedback/         # Feedback management
│   │   │   │       ├── Pending/     # Pending feedback folder
│   │   │   │       │   └── FeedbackForm/  # Feedback form screen
│   │   │   │       ├── Completed.tsx      # Completed feedback screen
│   │   │   │       └── Search.tsx         # Search feedback screen
```

```
│   │   │   ├── profile/            # User profile
│   │   │   │   └── allBadges/      # All badges screen
│   │   │   └── settings/           # Settings screens
│   │   │       ├── About.tsx       # About screen
│   │   │       ├── Edit.tsx        # Edit profile screen
│   │   │       └── Help.tsx        # Help screen
│   │   └── Login.tsx               # Authentication screen
│
├── stores/                         # MobX state stores
│   ├── authStore.ts                # Authentication state
│   ├── feedbackStore.ts            # Feedback data
│   ├── profileStore.ts             # User profile
│   ├── settingsStore.ts            # User preferences
│   └── notificationsStore.ts       # Notification management
│
├── components/                     # Reusable UI components
│   ├── ui/                         # Buttons, Inputs, Cards
│   ├── lists/                      # List items, CourseRow
│   └── modals/                     # Confirm, Edit feedback
│
├── hooks/                          # Custom React hooks
│
├── api/                            # API client with interceptors
│   └── client.ts
│
├── constants/                      # Colors, phase names, badges
│
├── types/                          # TypeScript definitions
│
├── utils/                          # Helper functions
│
└── assets/                         # Images, icons, fonts
```

10.2 State Management with MobX

MobX stores manage application state reactively:

```
export class RootStore {
apiClient: ApiClient;
authStore: AuthStore;
profileStore: ProfileStore;
feedbackStore: FeedbackStore;
```

```
settingsStore: SettingsStore;
notificationsStore: NotificationsStore;

constructor() {
makeAutoObservable(this, {}, { autoBind: true });
this.apiClient = new ApiClient(
process.env.EXPO_PUBLIC_API_URL || "http://localhost:8000",
);

this.notificationsStore = new NotificationsStore(this);
this.settingsStore = new SettingsStore(this);
this.authStore = new AuthStore(this);
this.profileStore = new ProfileStore(this);
this.feedbackStore = new FeedbackStore(this);
this.apiClient.setOnUnauthorized(() => {
this.authStore.handleUnauthorized();
});
}
}
```

10.3 API Client with Interceptors

Axios interceptors handle token refresh automatically:

```
api.interceptors.response.use(

  (response) => response,

  async (error) => {

    if (error.response?.status === 401 && !originalRequest._retry) {

      await refreshToken();

      return api(originalRequest);

    }

    return Promise.reject(error);

  }
```

```
);
```

10.4 Notification System

Local notifications are scheduled for:

- **Deadline reminders**: Day of deadline (9 AM)
- **Last chance**: 1 day before deadline (5 PM)
- **Early bird**: On start date to encourage early submission
- **Achievements**: Immediately when badge earned
- **Weekly reminders**: Monday at 10 AM

10.5 Badge System Implementation

Badges are calculated entirely on the frontend from completed feedback data, eliminating redundant database tables:

```
const badgeRules = {
  first_feedback: ({ total }) => ({ earned: total >= 1 }),
  feedback_rookie: ({ total }) => ({ earned: total >= 5 }),
  early_bird: ({ early_bird }) => ({ earned: early_bird }),
  // ... 42 total badges
};

const ctx = buildBadgeContext(completedFeedbacks);
this.userBadges = this.userBadges.map(badge =>
  updateBadgeProgress(badge, ctx)
);
```

10.6 Calendar Integration

The home screen features an interactive calendar marking dates with pending deadlines, using `react-native-calendars` with multi-period marking for courses with overlapping feedback periods.

placeholder

10.7 Theme Support

ThemeProvider wraps the application, supporting light/dark modes with system preference detection:

```
const Colors = {
  light: { NAVY: "#003A5D", BACKGROUND: "#F8F9FA" ,...},
  dark: { NAVY: "#1E3A5F", BACKGROUND: "#121212" ,...}
};
```

---

11. Web Dashboard

11.1 Project Structure

```
dashboard/src/
├── api/
| └── `client.ts` # HTTP client / interceptors
├── components/
| ├── `common/` # Header, Footer, Buttons
| ├── `charts/` # Histogram, LikertBar, Pie
| └── `forms/` # StudentForm, CourseForm
├── pages/ # Page-level feature folders
| ├── `Auth/` # Login, ForgotPassword
| ├── [Dashboard] # Overview, Widgets
| ├── `Courses/` # Courses list
| | └── `CourseDetails/` # Course Details Page
| ├── `Students/` # Student list & profile
| | └── `StudentDetails/` # Student Details Page
| └── `Reports/` # Export, Analytics
```

```
├── config/
│ └── `LayoutConfig.tsx` # Layout + routes config
├── constants/ # Colors, feedbackConfig
├── hooks/ # Reusable hooks (useFetch, useAuth)
├── layout/ # `Layout.tsx`, `Layout.css`
├── routes/ # ProtectedRoute, route helpers
├── types/ # TypeScript interfaces & types
├── utils/ # Helpers & formatting functions
├── assets/ # Static images & icons
└── tests/ # Unit / integration tests for src
```

11.2 Key Features

Dashboard Overview: Displays key metrics including total courses, students, feedback submitted, and completion rates using Ant Design and Statistic components.

Course Management: Table view with sorting, filtering, and search capabilities. Course details page shows enrollment list and feedback statistics.

Student Management: View and add student profiles with enrolled courses and feedback completion status.

Anonymous Feedback: Administrators can browse submitted feedback without student identification, maintaining privacy while providing course insights.

Bar Chart Analytics: Using `recharts` to visualize feedback response distribution and reveal polarization/ skewness/ trends

11.3 Protected Routes

Authentication is enforced through a wrapper component;

```
const ProtectedRoute = ({ children }) => {
  const token = localStorage.getItem('authToken');
  if (!token) return <Navigate to="/login" />;
  return children;
};
```

As well as Refresh mechanism using refresh tokens

```
if (error.response?.status === 401 && !originalRequest._retry) {

originalRequest._retry = true;



try {

const refreshToken = localStorage.getItem("refreshToken");

const response = await axios.post("/auth/refresh",{refreshToken,},);
const { authToken, refreshToken: newRefreshToken } = response.data;

localStorage.setItem("authToken", authToken);
localStorage.setItem("refreshToken", newRefreshToken);

client.defaults.headers.common["Authorization"] = `Bearer ${authToken}`;
originalRequest.headers.Authorization = `Bearer ${authToken}`;

return client(originalRequest);
} catch (refreshError) {
// Refresh failed - redirect to login
localStorage.removeItem("authToken");
localStorage.removeItem("refreshToken");
window.location.href = "/login";
return Promise.reject(refreshError);

}
```

11.4 CORS Configuration

Backend CORS configured to accept requests from both mobile app and web dashboard origins:

```


const allowedOrigins = [

"http://localhost:3000", // Web dev

"http://localhost:8081", // Expo web

```

```
"exp://localhost:8081", // Expo dev

process.env.MOBILE_URL, // Expo production

process.env.FRONTEND_URL, // Production web

];
```

---

12. Security Implementation

12.1 Authentication Security

- Passwords hashed with bcrypt (salt rounds = 10)
-
- JWT tokens with short expiration (15 minutes)
- Refresh token rotation on each refresh
- HTTP-only cookies not used (mobile apps use SecureStore)
- Rate limiting on login attempts

12.2 Data Protection

- Sensitive data (tokens, biometric preference) stored in Expo SecureStore
- Nonsensitive preferences in AsyncStorage
- SQL injection prevention via parameterized queries
- Input validation on all API endpoints
- CORS restrictions to allowed origins

12.3 Biometric Authentication

Biometric login is implemented as a convenience layer that uses stored refresh tokens:

1. User enables biometrics → stores preference
2. On subsequent logins → device biometric prompt
3. On success → retrieve refresh token → call refresh endpoint → issue new auth token

```
const result = await LocalAuthentication.authenticateAsync({

  promptMessage: `Authenticate with ${biometricType}`
```

```
});

if (result.success) {

  await biometricLogin();

}
```

12.4 Refresh Token Protection

The refresh token rotation strategy provides automatic token theft detection:

- Each refresh generates a new refresh token
- Old token is invalidated immediately
- If an old token is reused, server locks the account
- Tokens stored encrypted in SecureStore

---

13. User Interface Design

13.1 Mobile Application UI

Color Scheme:

- Navy (`#003A5D`) - Primary AUA Navy color
- Saffron (`#EEBC03`) - Accent and highlights, taken from AUA website
- White (`#FFFFFF`) - Backgrounds and cards

Typography: System defaults with consistent weight hierarchy

Key Screens:

1. **Login Screen**: Email/password inputs with biometric button
2. **Home Screen**: Calendar with marked deadlines, pending count badge
3. **Feedback Form**: Multi-step with progress indicator, star ratings, and text inputs
4. **Search Feedback Screen:** Course information browser with modals of aggregated statistics for every course
5. **Profile Screen**: User information, badge gallery, settings access

13.2 Web Dashboard UI

- Ant Design components for professional appearance

- Responsive tables with sorting and filtering
- Card-based metric displays
- Modal forms for Insertion operations
- Various charts for visualising and analysing data

13.3 Accessibility Considerations

- Sufficient color contrast
- Touch targets ≥44pt on mobile
- Semantic HTML in dashboard
- Screen reader support

13.4 Responsive Design

The web dashboard adapts to different screen sizes using Ant Design's grid system and responsive table configurations with horizontal scroll on smaller displays.

---

14. Challenges and Lessons Learned

14.1 Token Refresh Race Conditions

Challenge: When implementing axios interceptors, multiple concurrent API calls after token expiration started triggering multiple refresh requests, causing token invalidation conflicts.

Solution: Implemented a request queue that pauses subsequent requests while refreshing, then retries them with the new token.

```
let isRefreshing = false;

let refreshQueue: Promise[] = [];



// Queue requests while refreshing

if (isRefreshing) {

  return new Promise((resolve) => {

    refreshQueue.push(() => resolve(api(originalRequest)));
```

```
  });

}
```

14.2 IOS and Push-Notifications

Challenge: For enabling push notifications (new feedback period opened, deadline reminders, badge notifications, etc.), IOS builds required paid Apple Developer account.

Solution: migration to local in-app notifications, simplification of notification system

```
import * as Notifications from "expo-notifications";

const  scheduleNotification = async (
type: NotificationType,
options: ScheduleNotificationOptions = {},
): Promise<string | null> => {

    const  notificationId = await
Notifications.scheduleNotificationAsync({
        content: { title, body, data: { type, ...options } },
        trigger:

      });
    return notificationId;
}
```

14.3 Centralized Error Handling

Implementing a `withErrorHandling` wrapper eliminated try/catch duplication across 40+ store methods and provided consistent user feedback.

14.4 Key Takeouts

- **Biometric authentication design:** Learned and implemented biometric authentication functionality
- **MobX:** Discovered, researched and replaced existing Redux state management with much simpler and more intuitive Mobx design
- **Expo development and production builds:** initially project was intended to set finished on app design level and demonstration was supposed to happen via Expo's Expo Go application. But with release of new Expo SDK and necessity to implement various new

features such as local notification system, widgets,etc, development, and as a result, Production building tools and processes have been studied and implemented.

---

15. Future Improvements

15.1 LLM Model For Open-Ended Feedback Processing

Integrating of an LLM model would allow process feedbacks in a new way and provide insights from open feedbacks like commonly used descriptions, comparative analysis, descriptions(positive/negative) that started/stopped appearing in recent feedbacks, etc.

15.2 Widgets Integration

Alongside mobile app development, the possibility of developing widgets(Calendar, pending feedbacks) has been considered. Nevertheless because of the time limits, that part of the project has been left inside comments and unfinished. If next update is considerable, this feature will make into it without any questions.

15.3 Phase-Specific Questions.

Original idea of feedback system redesign included not only addition of new phases for evaluation, but also phase-specific questions in the forms and in some scenarios even phase-dependant different feedback forms to raise interest and encourage engagement and participation. Not only that idea has not been dismissed, but also it is being looked forward to be brought into life by the development team in near future.

15.4 Push Notifications

Implement remote push notifications for new feedback period announcements and deadline reminders. Requires enrollment in Apple Developer Program ($99/year) for iOS push capability.

15.5 Instructor Dashboards

Currently the role-check and role based access model technically exists and works in the system. Nevertheless creation of sophisticated role-based dashboards for instructors is considered, to allow them view aggregated feedback for their courses only, with student anonymity preserved.

15.6 Advanced Analytics

- Time-series trend analysis of course ratings over multiple semesters
- Comparative analytics between courses in same department
- Comparative analytics between same courses with different instructors(differing sections)

15.7 Export Functionality

Generate PDF/Excel reports of course feedback for instructors and department chairs.

15.8 Bulk import options

Building means of integrating existing data(courses, enrollments) into dashboard/app ecosystem

15.9 Performance Optimization

- Implement pagination for feedback history (currently loads all)
- Virtualized lists for long course catalogs
- Redis caching for frequently accessed analytics

15.10 CI/CD Pipeline

Automated testing and deployment using GitHub Actions with EAS for mobile builds and Render/Railway for backend.

---

16. Conclusion

CourseCritique successfully delivers a comprehensive, engaging, and secure course evaluation system redesign, including platforms for collecting and analyzing student course feedback, innovations in the evaluation form and new ways to look at the huge data that is collected semi-annually.

The project demonstrates the effective integration of modern technologies including React Native for cross-platform mobile development, TypeScript for type safety, MobX for state management, and PostgreSQL for robust data persistence.

The phased feedback structure (Add/Drop, Midterm, Finals) aligns with the natural academic calendar, capturing student experiences at meaningful intervals. The gamification system with 42 achievement badges across four categories is designed to provide motivation for consistent and high-quality participation and to prove that not everything academical has to be formal and mandatory. The privacy-focused design (anonymous feedback display) balances institutional needs for course improvement data with student confidentiality.

From a technical perspective, the project addressed several complex challenges including secure JWT authentication with refresh token rotation, platform-specific notification scheduling, and efficient JSONB data storage for flexible response schemas. The separation of authentication (`auth_users`) from user profiles (`profile_users`) provides a foundation for future role expansion (instructors, department chairs).

The administrative web dashboard offers comprehensive oversight through interactive tables, visual analytics with valuable insights, and course/student management tools. Throughout the whole process the author's aim was to create a technology that is fun and engaging to use not only from one side, but also from another. At the same time one thing was kept in mind: a technology that not only displays data, but also helps the user to use it. The responsive design ensures accessibility across different devices and screen sizes.

CourseCritique represents a production-ready solution that modernizes course evaluation processes in higher education. Its modular architecture, comprehensive testing, and security-conscious design provide a solid foundation for future enhancements including push notifications, instructor dashboards, and advanced analytics.

---

17. References

1. React Native Documentation. (2024). *Core Components and APIs*. https://reactnative.dev/docs/getting-started
2. Expo Documentation. (2025). *Expo SDK 55*. https://docs.expo.dev/
3. MobX Documentation. (2024). *State Management Library*. https://mobx.js.org/
4. PostgreSQL Documentation. (2024). *JSONB Types and Indexing*. https://www.postgresql.org/docs/current/datatype-json.html
5. JWT.io. (2024). *JSON Web Tokens Introduction*. https://jwt.io/
6. Ant Design Documentation. (2025). *React UI Components*. https://ant.design/
7. React Native Calendars. (2024). *Calendar Component*. https://github.com/wix/react-native-calendars
8. Expo Notifications. (2025). *Local and Push Notifications*. https://docs.expo.dev/versions/latest/sdk/notifications/
9. Auth0. (2024). *Refresh Token Rotation*. https://auth0.com/docs/secure/tokens/refresh-tokens/refresh-token-rotation
10. OWASP Foundation. (2024). *Authentication Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
11. Miller, E. (2009). *How not to sort by average rating*. Evan Miller. https://www.evanmiller.org/how-not-to-sort-by-average-rating.html
12. Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian data analysis* (3rd ed.). CRC Press.
13. Hoff, P. D. (2009). *A first course in Bayesian statistical methods*. Springer.

Appendix A: **Complete API Documentation** (Available via Swagger UI at `/api-docs`)

Appendix B: Badge Achievement Criteria (42 badges with calculation logic in `constants/badgeRules.ts`)